

Chat App System Design

WhatsApp / Messenger — Complete HLD Notes

WebSocket · User Mapping · Presence System
Cassandra · Group Chat · Full Architecture Diagram

TOPICS COVERED

1. Requirements & Scale Estimation

2. Protocols — Polling vs WebSocket

3. Message Flow — Online & Offline

4. User Mapping & Chat Servers

5. Database Design (Cassandra)

6. Group Chat Design

7. Presence System

8. Full Landscape HLD Diagram

Table of Contents

1. Requirements & Scale Estimation	3
2. Architecture — Client-Server vs P2P	3
3. Communication Protocols	3
4. Chat Servers & User Mapping	4
5. Message Flow — Online & Offline	5
6. Database Design (Cassandra/NoSQL)	5
7. Group Chat Design	6
8. User Authentication	6
9. Presence System — Online/Offline/Last Seen	6
10. Component Summary Table	7
11. Quick Revision & Interview Questions	7
12. Full Landscape System Architecture	8

1. Requirements & Scale Estimation

Functional Requirements

Feature	Description
One-to-One Messaging	Send/receive text messages between two users in real-time
Group Messaging	Create groups, add members, broadcast messages to all members
Online/Offline Status	Show real-time presence and last-seen timestamp
Message History	Store and retrieve past messages
User Authentication	Secure login with session management

Non-Functional Requirements

Requirement	Target	Implication
Scalability	Billions of messages/day	Horizontal sharding — NoSQL
Low Latency	< 100ms delivery	WebSocket + in-memory cache
High Availability	99.99% uptime	Distributed servers, no SPOF
Reliability	No message loss	Persist to DB, ACK mechanism

Back-of-the-Envelope Estimation

Parameter	Value	Calculation
Total users	2 billion	—
Daily Active Users (DAU)	50 million	—
Messages per user/day	40	10 messages x 4 recipients
Total messages/day	2 billion	50M x 40
Message size	100 bytes	Average text message
Daily storage	200 GB	2B x 100 bytes
10-year storage	~730 TB	200 GB x 365 x 10

2. Architecture — Client-Server vs Peer-to-Peer

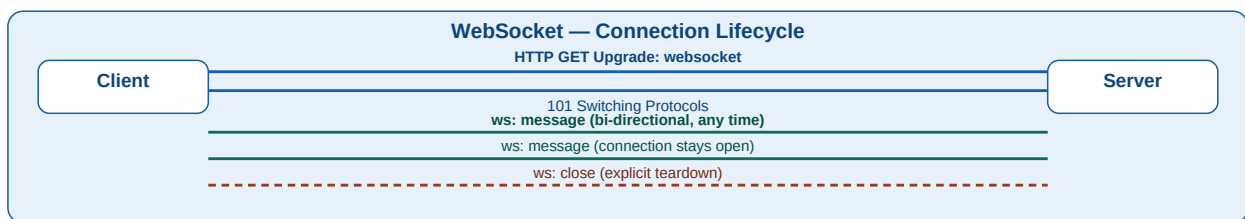
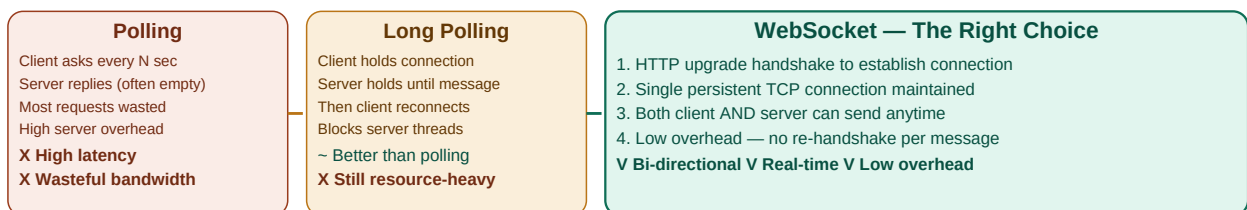
Aspect	Peer-to-Peer	Client-Server (Chosen)
How it works	Direct IP-to-IP connection	Users connect to chat servers
Chat history	Not stored anywhere	Stored in central DB
Group chat	Requires all IPs	Server manages group state

Aspect	Peer-to-Peer	Client-Server (Chosen)
Scalability	Poor — requires all IPs	Excellent — horizontal scaling
Offline messages	Impossible	Persisted to DB, delivered on reconnect
Verdict	Not suitable	Used by WhatsApp, Messenger, Discord

3. Communication Protocols

HTTP's request-response model works for sending messages but fails for receiving them — the server cannot initiate communication. Three alternatives exist:

Communication Protocols — Polling vs WebSocket



Polling wastes bandwidth · Long Polling blocks threads · WebSocket: persistent bi-directional connection for real-time messaging

WebSocket begins as HTTP (port 80/443), client sends "Upgrade: websocket" header.

Server responds with "101 Switching Protocols" — connection is now WebSocket.

Single TCP connection stays open — both sides can send frames at any time.

Used for: sending AND receiving messages, presence heartbeats, typing indicators, read receipts.

4. Chat Servers & User Mapping Service

Multiple chat servers handle WebSocket connections. Each chat server maintains connections to a subset of users. The **User Mapping Service** (implemented via Zookeeper or custom service) answers the critical question: *which chat server is User B connected to?*

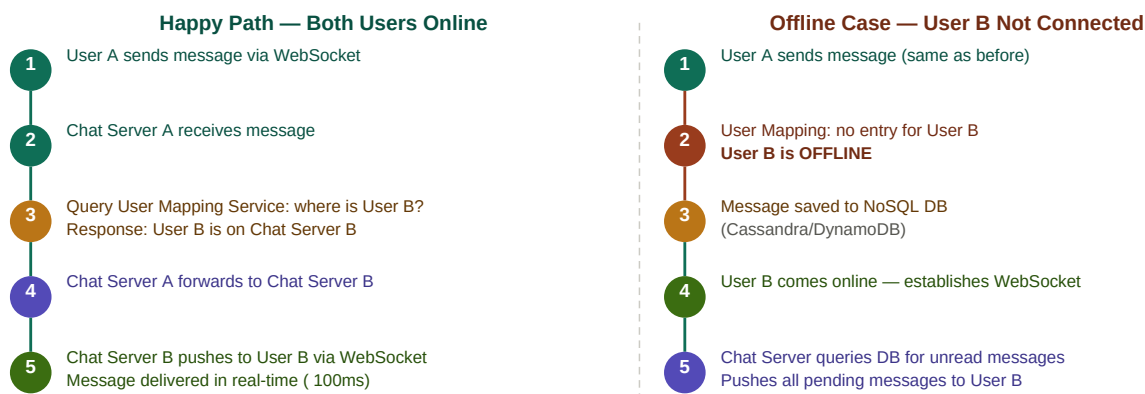
Component	Responsibility	Technology
Load Balancer	Routes incoming connections to chat servers	NGINX, HAProxy, AWS ALB
Chat Server	Maintains WebSocket connections, routes messages	Scalable stateful server
User Mapping Service	userID -> chatServerIP mapping	Zookeeper, etcd, custom service
User Login Service	Authenticates users, issues tokens, updates mapping	HTTP/S, OAuth, JWT

User Mapping Service — How It Works

- When User B connects, their chat server registers: mapping[userB] = chatServerB_IP
- When Chat Server A needs to send to User B: query mapping -> get chatServerB_IP
- Forward message from Chat Server A to Chat Server B
- Chat Server B delivers via User B's open WebSocket
- If User B is offline: no entry in mapping -> save to DB instead

5. Message Flow — Online & Offline

Message Flow — Online Offline Cases



Data Model — Messages Table				
message_id	sender_id	receiver_id	timestamp	content
ts_uuid_1234	userA	userB	2024-01-01T10:00	Hey, how are you?
Partition key: sender_id + receiver_id Sort key: message_id (timestamp-based) NoSQL: Cassandra / DynamoDB				

Happy path: 5-hop WebSocket delivery in real-time · Offline: persist to Cassandra, deliver on reconnect

6. Database Design — NoSQL (Cassandra)

Why NoSQL (Cassandra)? Chat messages are written at massive scale (2B/day) and read in time-sorted order per conversation. Cassandra's column-family model with partition keys and time-ordered sorting is a perfect fit.

Column	Type	Purpose
message_id	UUID (time-based)	Unique ID + ordering within partition
sender_id	String	Who sent the message
receiver_id	String	Who receives (or group_id for group)
timestamp	Timestamp	When sent — also used for sorting
content	Text	Message body (text/media URL)
status	Enum	SENT / DELIVERED / READ

Partition key: (sender_id, receiver_id) — all messages in a conversation go to the same partition.

Sort key: message_id (timestamp-based UUID) — messages returned in chronological order automatically.

Why not SQL: no complex joins needed, horizontal sharding is automatic, write throughput is massive.

Discord uses Cassandra. Facebook Messenger used HBase (similar concept). WhatsApp uses custom store.

7. Group Chat Design

Group chat reuses the same architecture but adds a **Group Management Service** to track membership. The `group_id` acts as the partition key instead of individual sender/receiver IDs.

- Group Management Service stores: `group_id`, `group_name`, member list, `created_at`, `admin_ids`
- When a message is sent to a group, the chat server queries Group Management for all member IDs
- For each member: query User Mapping to find their chat server
- Fan out the message to each member's chat server via WebSocket
- Offline members: message stored with `group_id` as partition key, delivered on reconnect
- For large groups (1000+ members): use a message queue (Kafka) to fan out asynchronously

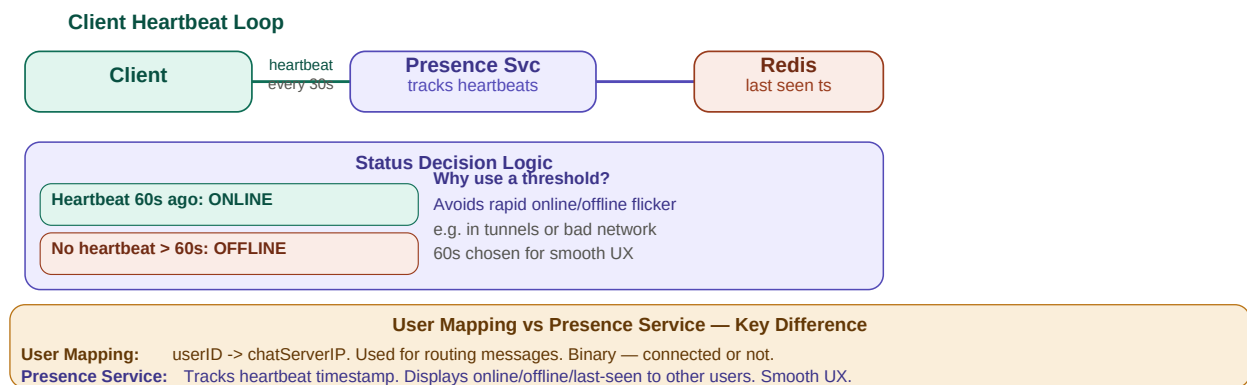
8. User Authentication

Authentication uses standard HTTP/S (not WebSocket) since it is a simple request-response flow. After login, a token (JWT or session ID) is issued and used to authenticate the subsequent WebSocket upgrade request.

- User sends credentials to User Login Service via HTTPS POST
- Login Service validates, issues JWT token
- Client uses JWT to authenticate WebSocket connection header
- Chat Server validates JWT before accepting WebSocket upgrade
- On successful login, User Mapping is updated: `mapping[userId] = thisChatServer`

9. Presence System — Online / Offline / Last Seen

Presence System — Online / Offline / Last Seen



Client sends heartbeat every 30s -> Presence Service -> Redis · No heartbeat for 60s = mark offline

Why Use a Threshold (Not Instant)?

Without a threshold, a user entering a tunnel (10s no network) would flash Offline and back Online. The 60-second threshold smooths this out — short interruptions don't change status. This is the **soft state** principle from BASE properties.

Aspect	User Mapping Service	Presence Service
Purpose	Route messages to correct chat server	Show online/offline status to other users

Aspect	User Mapping Service	Presence Service
Data stored	userID -> chatServerIP	userID -> last_heartbeat_timestamp
Updated when	User connects/disconnects	Every 30 seconds (heartbeat)
Storage	Zookeeper / etcd	Redis (in-memory, fast reads)
Failure impact	Message routing fails	Wrong status shown (temporary)

10. Component Summary

Component	Role	Technology
Client	WebSocket connection, heartbeats, UI	Mobile/Web app
Load Balancer	Route connections to chat servers	NGINX, HAProxy, AWS ALB
Chat Server	WebSocket mgmt, message routing	Node.js, Go, Erlang
User Mapping	userID -> serverIP routing table	Zookeeper, etcd
User Login	Auth, sessions, token issuance	HTTP/S, OAuth, JWT
Group Mgmt	Group creation, membership, metadata	Microservice + NoSQL
Presence Service	Heartbeat tracking, online/offline status	Custom + Redis
NoSQL DB	Chat history, offline messages	Cassandra, DynamoDB
Redis/Cache	Presence data, hot messages, sessions	Redis Cluster
User DB	User profiles, auth data	MySQL or MongoDB

11. Quick Revision & Interview Questions

Quick Revision

- > Use Client-Server, not P2P — P2P has no chat history, no group support, not scalable.
- > WebSocket for real-time: HTTP upgrade handshake -> persistent bi-directional TCP connection.
- > Polling: wasteful. Long Polling: better but blocks threads. WebSocket: correct choice.
- > Chat Server: stateful — maintains WebSocket connections for its assigned users.
- > User Mapping Service: userID -> chatServerIP. Queried on every message send.
- > Happy path: A -> Chat Server A -> User Mapping -> Chat Server B -> B (WebSocket).
- > Offline path: User Mapping has no entry -> save to Cassandra -> deliver on B's next login.
- > Cassandra partition key: (sender_id, receiver_id). Sort key: timestamp-based message_id.
- > Group chat: Group Mgmt Service stores members. Fan out to each member's chat server.
- > Presence: heartbeat every 30s from client -> Redis. No heartbeat for 60s = offline.
- > User Mapping != Presence. Mapping = routing. Presence = UX status display.
- > Scale: 50M DAU x 40 msgs = 2B msgs/day. At 100 bytes each = 200 GB/day storage.
- > Cassandra chosen for: horizontal scaling, high write throughput, time-sorted queries.
- > Redis stores presence data for O(1) lookups of online/offline status.

Interview Questions

1. Design a chat system like WhatsApp. Walk through the full architecture.
2. Why WebSocket over HTTP for a chat application?
3. What is the User Mapping Service and why is it needed?

- 4. Walk me through what happens when User A sends a message to offline User B.
- 5. Why choose Cassandra over MySQL for storing chat messages?
- 6. How would you design group messaging for groups of 1000+ members?
- 7. What is the difference between the User Mapping Service and the Presence Service?
- 8. How does the presence system avoid showing users as offline during brief network drops?
- 9. How would you scale the chat servers to handle 50 million concurrent users?
- 10. How would you add typing indicators and read receipts to this design?
- 11. What is the partition key in Cassandra for this chat schema and why?

WhatsApp-Style Chat App — Full System Architecture

